

//Program Requires RadioHead Library from Mike McCauley

#include <RH_ASK.h>

#include <SPI.h>

#include <ServoTimer2.h>

RH_ASK rf_driver;

// Motor A connections

int enA = 3;

int in1 = 4;

int in2 = 5;

// Motor B connections

int enB = 8;

int in3 = 6;

int in4 = 7;

//Payload connection

int payloadPin = 9;

//RF Transmitter connection

int transmitterPin = 11;

//RF Distance to go variable

int distanceToGo = 0;

//Payload connection

int servoPin = 9;

int angle = 0;

ServoTimer2 myServo;

```
void setup() {  
    // Set all the motor control pins to outputs  
    pinMode(enA, OUTPUT);  
    pinMode(enB, OUTPUT);  
    pinMode(in1, OUTPUT);  
    pinMode(in2, OUTPUT);  
    pinMode(in3, OUTPUT);  
    pinMode(in4, OUTPUT);  
  
    // Turn off motors - Initial state  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
  
    // Payload - Initial State ON  
    digitalWrite(payloadPin, HIGH);  
  
    //Initialize ASK Object  
    rf_driver.init();  
    Serial.begin(9600);  
}  
  
void loop() {  
    distanceToGo=0;
```

```

    recieveRFMessage();

    Serial.println(distanceToGo);

    if(distanceToGo > 0) {

        delay(10000);

        driveDistanceVariable();

        delay(1000);

        dropPayload();

    }

}

// Recieve buffer message function
void recieveRFMessage(){

    uint8_t buf[15];

    uint8_t buflen = sizeof(buf);

    // Check if received packet is correct size

    if (rf_driver.recv(buf, &buflen))

    {

        // Message received with valid checksum

        Serial.print("Message Received: ");

        Serial.println((char*)buf);

        // Distance To Go Based on recieved message placed on variable

        sscanf(buf, "%*c%d", &distanceToGo);

    }

}

```

```
void driveDistanceVariable(){  
    // Turn on motors  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
  
    analogWrite(enA, 255);  
    analogWrite(enB, 255);  
  
    delay(20);  
  
    // Speed analog values for motors, change for testing  
    int straightSpeedA = 101;  
    int straightSpeedB = 100;  
    analogWrite(enA, straightSpeedA);  
    analogWrite(enB, straightSpeedB);  
  
    // Run function for X seconds at 1 ft/s  
    int timeToRun = distanceToGo * 250;  
    delay(timeToRun);  
  
    // Now turn off motors  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);
```

```
digitalWrite(in3, LOW);  
digitalWrite(in4, LOW);  
  
}  
  
// Payload Drop Function  
void dropPayload() {  
    myServo.attach(servoPin);  
    // Sweep the servo from 0 to 180 degrees  
    for(angle = 0; angle <= 110; angle += 22) {  
        myServo.write(angle); // Set the servo position  
        Serial.print("Angle: ");  
        Serial.println(angle);  
        delay (5);  
        // Wait 15ms for the servo to reach the position  
    }  
  
    delay (1000);  
    // Sweep back from 180 to 0 degrees  
    for(angle = 110; angle >= 0; angle -= 22) {  
        myServo.write(angle);  
        Serial.print("Angle: ");  
        Serial.println(angle);  
        delay(5);  
    }  
}
```

```
delay (1000);
```

```
}
```